

KUBERNETES

Pod

1. Pod là gì? Định nghĩa chính thức

Pod là đơn vị triển khai nhỏ nhất và đơn giản nhất (smallest deployable unit) trong Kubernetes.

Một Pod đại diện cho một nhóm gồm một hoặc nhiều container (thường là Docker container) được chạy cùng nhau trên cùng một Node. Các container trong cùng Pod chia sẻ:

- Không gian mạng (network namespace): Chúng có chung IP địa chỉ và có thể giao tiếp với nhau qua localhost.
- Không gian lưu trữ (shared storage/volumes): Các volume được gắn vào Pod và chia sẻ giữa các container.
- Thông tin chạy (specification): Cách chạy, restart policy, v.v.

Pod được ví như một "logical host" ứng dụng cụ thể (application-specific logical host) – giống như một máy ảo nhỏ hoặc một nhóm container gắn kết chặt chẽ (tightly coupled), hoạt động như một đơn vị duy nhất.

2. Tại sao Kubernetes dùng Pod thay vì quản lý container trực tiếp?

Kubernetes không quản lý container riêng lẻ mà quản lý qua Pod vì:

- Pod là đơn vị nguyên tử (atomic unit) để scheduler quyết định đặt lên Node nào.
- Đảm bảo các container liên quan chặt chẽ (ví dụ: container chính + sidecar logging, proxy, monitoring) luôn co-located (chạy cùng Node), chia sẻ tài nguyên mạng và lưu trữ mà không cần cấu hình phức tạp.
- Cho phép mô hình "one-container-per-Pod" (phổ biến nhất) hoặc multi-container khi cần (nhưng ít dùng hơn).

Nếu chỉ nói "Pod giống server" thì chưa chính xác – nó không phải là server vật lý (Node), mà là môi trường chia sẻ (shared environment) hoặc máy chủ ứng dụng logic.

3. Pod nằm trong kiến trúc Kubernetes như thế nào?

- Node (worker node) chứa nhiều Pod.
- Mỗi Pod có IP riêng trong mạng Pod (Pod network), thuộc về cụm Kubernetes.
- Pod thuộc về một Namespace (không gian tên) để phân tách dự án, môi trường (dev, prod...).
- Pod là ephemeral (tạm thời, không bền vững): Khi Pod chết, Kubernetes không tự tạo lại trừ khi dùng controller cao cấp hơn (Deployment, ReplicaSet...).

4. Các mô hình sử dụng Pod phổ biến

- Single-container Pod (mô hình phổ biến và được khuyến nghị): Chỉ 1 container chính → dễ quản lý, debug, scale, tránh phụ thuộc chéo.

- Multi-container Pod: 1 container chính + các sidecar (ví dụ: logging, proxy, config sync). Các container này phải gắn kết chặt chẽ, chia sẻ lifecycle (chết cùng nhau).

Lưu ý thực tế quan trọng (hay hỏi phỏng vấn): Nên ưu tiên một container chính mỗi Pod để:

- Giảm sự phụ thuộc (nếu một container lỗi không ảnh hưởng toàn bộ).
- Dễ debug, troubleshoot.
- Tránh lỗi lan rộng (cascading failure).

5. Pod có IP và mạng như thế nào?

- Mỗi Pod được cấp một địa chỉ IP riêng trong cụm (không phải IP của Node).
- Tất cả container trong Pod dùng chung IP đó → giao tiếp nội bộ qua localhost + port.
- Container bên ngoài Pod giao tiếp với Pod qua IP của Pod + port được expose.

6. Pod Lifecycle (vòng đời) – Phần hay hỏi

Pod có các phase chính:

- Pending: Chờ scheduler chọn Node, kéo image, hoặc chuẩn bị tài nguyên.
- Running: Ít nhất một container đang chạy bình thường.
- Succeeded: Tất cả container hoàn thành thành công (thường dùng cho Job).
- Failed: Có container thất bại và không thể tiếp tục.
- Unknown: Không kết nối được với Node (lỗi kubelet hoặc mạng).

Pod có restart policy (Always – mặc định, OnFailure, Never) quyết định container có restart khi crash không.

7. Pod vs các khái niệm khác (hay so sánh trong phỏng vấn)

- Pod vs Container: Container là đơn vị chạy code, Pod là wrapper bao quanh 1+ container + tài nguyên chia sẻ.

- Pod vs Node: Node là máy worker vật lý/ảo, Pod chạy trên Node.

- Pod vs Deployment/ReplicaSet: Pod là instance đơn lẻ, ephemeral.

Deployment/ReplicaSet quản lý nhiều Pod, tự heal, scale, rolling update.

- Pod vs Service: Service là abstraction để expose Pod (stable IP/DNS), Pod có IP thay đổi khi recreate.

8. Một số điểm lưu ý thực tế & phỏng vấn

- Pod không được dùng độc lập lâu dài trong production → không tự heal, không scale.

- Không nên chạy nhiều container không liên quan trong cùng Pod → tăng rủi ro, khó debug.

- Pod là nền tảng cho hầu hết workload (Deployment, StatefulSet, DaemonSet, Job... đều tạo Pod).

- Khi phỏng vấn, tránh nói "Pod là server" – hãy nhấn mạnh "Pod là nhóm container chia sẻ mạng và lưu trữ, là đơn vị nhỏ nhất Kubernetes quản lý".

Deployment

1. Deployment là gì? Định nghĩa chính thức

Deployment là một controller (workload controller) cấp cao trong Kubernetes, dùng để quản lý các Pod một cách declarative (mô tả trạng thái mong muốn).

Nó quản lý một tập hợp các Pod giống hệt nhau (replicas) để chạy một ứng dụng stateless (không lưu trạng thái). Deployment đảm bảo ứng dụng luôn chạy đúng số lượng replicas mong muốn, và hỗ trợ cập nhật ứng dụng một cách an toàn, có kiểm soát.

Deployment không tạo Pod trực tiếp, mà tạo ReplicaSet để quản lý số lượng replicas của Pod. ReplicaSet là lớp thấp hơn, chỉ đảm bảo số lượng Pod mong muốn, còn Deployment thêm các tính năng cao cấp hơn.

2. Tại sao cần Deployment? (So với Pod hoặc ReplicaSet độc lập)

- Pod độc lập là ephemeral (tạm thời): Nếu Pod chết, không tự tạo lại; không scale, không update version.
- ReplicaSet chỉ đảm bảo số lượng replicas (self-healing cơ bản), nhưng không hỗ trợ rolling update, rollback, versioning.
- Deployment là abstraction cao cấp nhất cho stateless workload:
 - Tự động tạo ReplicaSet mới khi update (ví dụ: thay đổi image).
 - Hỗ trợ scaling (tăng/giảm replicas).
 - Self-healing: Nếu Pod lỗi, tự tạo Pod mới để đạt replicas mong muốn.
 - Rolling updates và rollback để update version mà không downtime (hoặc downtime tối thiểu).
 - Lưu lịch sử revisions (phiên bản) để rollback nhanh.

Deployment là tài nguyên phổ biến nhất để triển khai ứng dụng thực tế trong production (stateless apps như web server, API).

3. Các tính năng chính của Deployment (hay hỏi phỏng vấn)

- Replicas management (Replica): Số lượng Pod mong muốn (desired replicas).

Deployment luôn duy trì đúng số này.

- Self-healing: Pod chết hoặc Node lỗi → Deployment tạo Pod mới thay thế.
- Scaling: Tăng/giảm replicas thủ công hoặc tự động (qua Horizontal Pod Autoscaler – HPA).
- Rolling updates: Cập nhật version mới (thay image, config...) dần dần, thay thế Pod cũ bằng Pod mới theo cách kiểm soát (zero-downtime nếu cấu hình tốt).
- Rollback: Quay về version trước nếu update lỗi (dựa trên revision history).

- Revision history: Lưu lịch sử các thay đổi (mặc định 10 revisions), cho phép rollback nhanh.
- Declarative: Bạn khai báo desired state, Deployment Controller điều chỉnh actual state.

4. Deployment Strategy (Chiến lược triển khai) – Phần quan trọng nhất

Deployment hỗ trợ 2 chiến lược chính (strategy.type):

RollingUpdate (mặc định và được khuyến nghị):

- Cập nhật dần dần (incremental).
- Tạo Pod mới với version mới, đồng thời scale down Pod cũ.
- Đảm bảo zero-downtime (hoặc downtime rất thấp) nếu cấu hình tốt.
- Các tham số chính:
 - maxSurge: Số Pod thừa tối đa (surge) trong quá trình update (số hoặc %).
 - maxUnavailable: Số Pod tối đa có thể unavailable (không sẵn sàng) trong quá trình.
- Ưu điểm: Không gián đoạn dịch vụ, an toàn cho production.
- Nhược điểm: Chậm hơn nếu image nặng hoặc nhiều replicas.

Recreate:

- Xóa toàn bộ Pod cũ trước, sau đó tạo Pod mới với version mới.
- Gây downtime (tất cả Pod unavailable trong lúc recreate).
- Ưu điểm: Nhanh, đảm bảo tất cả Pod cùng version (không có Pod cũ lẫn mới).
- Nhược điểm: Không phù hợp production (chỉ dùng cho dev/test, job, hoặc app chấp nhận downtime).

Trong thực tế: RollingUpdate dùng cho hầu hết ứng dụng production; Recreate dùng khi cần clean state hoàn toàn hoặc tốc độ quan trọng hơn availability.

5. Deployment vs các khái niệm khác (hay so sánh phỏng vấn)

- Deployment vs Pod: Pod là instance đơn lẻ, ephemeral. Deployment quản lý nhiều Pod replicas, tự heal, scale, update.
- Deployment vs ReplicaSet: ReplicaSet chỉ quản lý replicas (số lượng Pod giống hệt). Deployment quản lý ReplicaSet, thêm rolling update, rollback, revision.
- Deployment vs StatefulSet: Deployment cho stateless (Pod giống hệt, interchangeable). StatefulSet cho stateful (Pod có identity riêng, thứ tự tạo/xóa, persistent storage – ví dụ database).
- Deployment vs DaemonSet: DaemonSet chạy 1 Pod trên mỗi Node (monitoring, logging agent). Deployment chạy replicas bất kỳ Node nào.

6. Deployment trong lifecycle và thực tế

- Deployment tạo ReplicaSet → ReplicaSet tạo Pod.
- Khi update (ví dụ: thay image), Deployment tạo ReplicaSet mới → scale up mới, scale down cũ.

- Service thường trỏ đến Deployment qua selector (labels) → traffic load balance đến các Pod của Deployment.
- Deployment phù hợp stateless apps (web, API, microservices).
- Không dùng Deployment cho stateful apps (dùng StatefulSet) hoặc one-per-node (DaemonSet).

7. Điểm lưu ý thực tế & phỏng vấn

- Deployment là nền tảng cho CI/CD: Update image → rolling update tự động.
- Luôn dùng RollingUpdate với maxSurge/maxUnavailable hợp lý (ví dụ: 25%) để tránh downtime.
- Revision history giúp rollback nhanh (kubectl rollout undo).
- Deployment không tự autoscaling (dùng HPA riêng).
- Khi phỏng vấn: Nhấn mạnh Deployment mang lại declarative updates, zero-downtime rolling updates, self-healing, rollback – đây là lý do Deployment mạnh mẽ hơn mô hình truyền thống (update thủ công trên server).

Service

1. Service là gì? Định nghĩa chính thức

Service là một abstraction (lớp trừu tượng) trong Kubernetes, dùng để expose (tiếp cận) một nhóm Pod (thường từ Deployment) qua một địa chỉ ổn định (IP hoặc DNS name). Pod có IP tạm thời (ephemeral) – khi Pod chết hoặc scale, IP thay đổi. Service giải quyết vấn đề này bằng cách cung cấp địa chỉ cố định (stable endpoint) và load balancing (phân phối traffic) đến các Pod backend phù hợp.

Ví dụ dễ hiểu:

Pod giống như các nhân viên bán bánh ở quầy (có thể thay đổi, di chuyển).

Service giống như người tư vấn hoặc bảng chỉ dẫn ở cửa tiệm: Khách (traffic) đến hỏi "quầy bánh ngọt đâu?", người tư vấn sẽ dẫn thẳng đến đúng quầy (các Pod), mà không cần biết nhân viên nào đang đứng ở đâu.

Service dùng selector (labels) để chọn nhóm Pod nào thuộc về nó, và tự động cập nhật khi Pod thay đổi.

2. Tại sao cần Service? (Mục đích chính)

Cung cấp địa chỉ ổn định (IP/DNS) cho ứng dụng, dù Pod thay đổi IP hay số lượng.

Load balancing tự động: Phân phối traffic đều đến các Pod sẵn sàng (ready).

Service discovery nội bộ: Các Pod khác trong cụm có thể gọi nhau qua tên DNS (ví dụ: my-service.my-namespace.svc.cluster.local).

Cho phép expose ứng dụng ra bên ngoài cụm (hoặc giữ nội bộ).

Tách biệt frontend (Ingress/Gateway) với backend (Pod), để kiểm soát traffic.

Không có Service, bạn không thể truy cập ổn định vào ứng dụng từ bên ngoài hoặc giữa các microservices.

3. Các loại Service chính (4 loại phổ biến – hay hỏi phỏng vấn)

Kubernetes có 4 loại Service chính (type), mỗi loại quyết định cách expose:

1. ClusterIP (mặc định, phổ biến nhất):

- Tạo IP nội bộ (cluster-internal IP) chỉ truy cập được từ bên trong cụm.
- Không expose ra ngoài (bên ngoài không thấy IP này).
- Dùng cho: Giao tiếp giữa các service nội bộ (ví dụ: frontend gọi backend, backend gọi database trong cùng cụm).
- Ví dụ tiệm bánh: Bảng chỉ dẫn bên trong tiệm – chỉ nhân viên và khách trong tiệm mới biết đường đến quầy.

2. NodePort:

- Expose Service ra bên ngoài qua port cố định (NodePort, thường 30000–32767) trên mọi Node (máy worker).
- Truy cập: <Node-IP>:<NodePort>.
- Vẫn có ClusterIP bên trong, NodePort chỉ là "cửa phụ" thêm.
- Dùng cho: Test nhanh, debug, hoặc expose tạm thời ra ngoài (không khuyến khích production vì khó kiểm soát, không an toàn).
- Ví dụ tiệm bánh: Mở cửa phụ trực tiếp từ đường vào quầy bánh ngọt – ai cũng đi thẳng được, nhưng không qua cửa chính (khó kiểm soát bảo mật).

3. LoadBalancer:

- Tạo external load balancer từ cloud provider (AWS ELB, GCP Load Balancer, Azure LB...).
- Tự động cấp IP công cộng (external IP) và phân phối traffic đến các Pod.
- Dùng cho: Expose ứng dụng ra internet production trên cloud (có load balancing thực thụ).
- Ví dụ tiệm bánh: Thuê người gác cổng lớn (load balancer) ở ngoài đường, dẫn khách vào tiệm an toàn và phân bổ đều đến các quầy.

4. ExternalName:

- Không proxy traffic, chỉ map Service đến tên DNS bên ngoài (external DNS name).
- Trả về CNAME record → traffic đi thẳng ra ngoài cụm.
- Dùng cho: Kết nối với dịch vụ bên ngoài (database SaaS, API external) mà không cần thay đổi code (dễ migrate).
- Ví dụ tiệm bánh: Khách hỏi bánh ngọt, nhân viên bảo "hết hàng ở đây, qua cơ sở ABC.com mua nhé" – redirect thẳng đến địa chỉ khác.

Lưu ý: Headless Service (không IP cluster) là biến thể đặc biệt (không type riêng), dùng khi cần truy cập trực tiếp từng Pod (StatefulSet).

4. Service trong thực tế & phỏng vấn

- ClusterIP + Ingress (bài sau) là cách chuyên nghiệp nhất cho production: ClusterIP nội bộ + Ingress làm "cửa chính" kiểm soát traffic, TLS, routing.
- LoadBalancer phổ biến trên cloud (AWS/GCP/Azure), nhưng tốn chi phí.
- NodePort chỉ dùng dev/test/debug – production tránh vì port cao, khó firewall, không an toàn.
- Service dùng DNS nội bộ (CoreDNS) để resolve tên → các Pod gọi nhau dễ dàng (microservices).
- Service gắn với Deployment qua selector (labels) → khi scale Deployment, Service tự động load balance đến Pod mới.

5. So sánh nhanh (hay hỏi phỏng vấn)

- ClusterIP: Nội bộ → an toàn, mặc định.
- NodePort: Nội bộ + expose qua Node port → test nhanh, không khuyến khích prod.
- LoadBalancer: Nội bộ + external LB + public IP → production cloud.
- ExternalName: Không proxy, redirect DNS → kết nối external.

Service là "cầu nối" giữa Pod (backend) và client (người dùng hoặc Pod khác), giúp ứng dụng ổn định dù Pod thay đổi.

Ingress

1. Ingress là gì? Định nghĩa đơn giản

Ingress là một tài nguyên Kubernetes (API object) dùng để quản lý truy cập từ bên ngoài (external traffic, thường là HTTP/HTTPS) vào các Service bên trong cụm K8s.

Nó hoạt động như cửa chính của tiệm bánh:

- Khách (client từ internet) đến cửa tiệm (Ingress).
- Người bảo vệ (Ingress Controller) kiểm tra và dẫn khách đến đúng quầy (Service → Pod) dựa trên quy tắc (rules): domain, đường dẫn URL, v.v.

Ingress không phải là loại Service, mà là một lớp Layer 7 (application layer) routing thông minh, hiểu về HTTP/HTTPS (host, path, header...), khác với Service chỉ làm Layer 4 (IP/port).

2. Tại sao cần Ingress? (Mục đích chính – hay hỏi phỏng vấn)

- Tiết kiệm IP và Load Balancer: Thay vì expose từng Service ra ngoài (dùng LoadBalancer/NodePort cho mỗi cái → tốn kém, phức tạp), bạn chỉ cần 1 entry point (1 IP công cộng) cho nhiều Service.
- Routing thông minh dựa trên HTTP/HTTPS:
 - Path-based routing: /api → backend API, /web → frontend.
 - Host-based routing: app1.example.com → Service A, app2.example.com → Service B.
- Tính năng Layer 7 mạnh mẽ:

- SSL/TLS termination (chấm dứt HTTPS tại Ingress).
 - Name-based virtual hosting (nhiều domain trên cùng IP).
 - Load balancing, rate limiting, IP whitelist/blacklist, rewrite URL, auth cơ bản, v.v.
 - Zero-downtime khi scale/update: Ingress tự động cập nhật khi Pod/Service thay đổi.
- Không có Ingress, bạn phải dùng nhiều LoadBalancer (tốn tiền trên cloud) hoặc NodePort (port cao, khó quản lý, không an toàn).

3. Ingress hoạt động như thế nào? (Mô hình chuẩn)

Mô hình flow điển hình (work flow thực tế):

Client (browser/internet) → External Load Balancer (cloud LB hoặc HAProxy/NGINX ngoài cụm) → Ingress Controller (Pod chạy trong cụm) → Ingress resource (rules) → Service (ClusterIP) → Pod (Deployment).

- Ingress resource: Chỉ là file khai báo rules (YAML).
- Ingress Controller: Là "người bảo vệ" thực sự – một Deployment chạy Pod (thường NGINX, Traefik, HAProxy...) – nó đọc rules từ Ingress resource và cấu hình proxy/load balancer tương ứng.

Không có Controller → Ingress vô nghĩa (404 error).

Ví dụ tiệm bánh:

- Ingress = cửa tiệm + bảng chỉ dẫn.
- Ingress Controller = người bảo vệ đứng cửa, đọc bảng chỉ dẫn và dẫn khách đúng quầy.
- Service = quầy bánh (ClusterIP nội bộ).
- Pod = nhân viên bán bánh.

4. Ingress Controller phổ biến (hay hỏi phỏng vấn)

Bạn phải cài một Controller để Ingress hoạt động. Các loại phổ biến (2024-2025):

- NGINX Ingress Controller (kubernetes/ingress-nginx): Phổ biến nhất, ổn định, nhiều annotations tùy chỉnh (upload size, auth, rewrite...).
- Traefik: Tự động discovery, dynamic config, dễ dùng cho microservices, hỗ trợ Let's Encrypt tự động.
- HAProxy Ingress: Hiệu suất cao, tốt cho traffic lớn.
- Contour, Ambassador, Istio Gateway (nâng cao hơn).

Trên cloud (GKE, EKS, AKS): Có thể dùng native LoadBalancer + Ingress, hoặc cài NGINX/Traefik.

On-premise: Thường dùng NGINX + external LB (HAProxy/NGINX ngoài cụm) để expose.

5. Ingress vs Service (so sánh quan trọng – hay hỏi)

Service:

- Layer 4 (IP/port).

- Expose Pod nội bộ (ClusterIP) hoặc ngoài (NodePort/LoadBalancer).
- Không hiểu HTTP (không route theo path/domain).
- Dùng cho mọi protocol (TCP/UDP).

Ingress:

- Layer 7 (HTTP/HTTPS).
- Route dựa trên host/path/header.
- Tiết kiệm: 1 IP cho nhiều Service.
- Cần Controller + thường kết hợp Service ClusterIP.

Tóm tắt: Service = "đường nội bộ trong tiệm", Ingress = "cửa chính + chỉ dẫn thông minh từ ngoài vào".

6. Ingress Class (lưu ý quan trọng)

- Một cụm có thể có nhiều Ingress Controller (ví dụ: NGINX cho web, Traefik cho API).
- Ingress resource phải chỉ định ingressClassName (hoặc annotation cũ) để biết dùng Controller nào.
- Không chỉ định → Controller mặc định (nếu có) hoặc lỗi.

7. Lưu ý thực tế & phỏng vấn

- Ingress chỉ hỗ trợ HTTP/HTTPS (không TCP/UDP thuần – dùng Service cho cái đó).
- Production: Luôn dùng HTTPS (TLS termination tại Ingress), kết hợp cert-manager cho Let's Encrypt.
- On-premise: Cần external LB (HAProxy/NGINX) expose Ingress Controller (thường NodePort).
- Cloud: LoadBalancer tự động cấp public IP cho Ingress Controller.
- Ingress là phần cuối của "expose application" flow: Deployment → Service (ClusterIP) → Ingress → External LB/Internet.

ConfigMap

1. ConfigMap là gì? Định nghĩa đơn giản

ConfigMap là một API object trong Kubernetes dùng để lưu trữ dữ liệu cấu hình không nhạy cảm (non-confidential) dưới dạng cặp key-value (chuỗi văn bản) hoặc toàn bộ file cấu hình.

Nó giống như một tệp cấu hình chung (dictionary) mà Kubernetes cung cấp sẵn:

- Bạn lưu các thiết lập như URL database, port, log level, tên file config... vào ConfigMap.

- Các Pod (container) có thể đọc dữ liệu này mà không cần nhúng cứng (hardcode) vào code hoặc Docker image.

Mục tiêu chính: Tách biệt cấu hình khỏi ứng dụng → dễ thay đổi config mà không cần rebuild image hoặc restart toàn bộ Pod (chỉ cần update ConfigMap và redeploy nếu cần).

2. Tại sao cần ConfigMap? (Mục đích chính – hay hỏi phỏng vấn)

- Linh hoạt và portable: Cấu hình môi trường (dev, staging, prod) khác nhau → chỉ thay ConfigMap, không sửa code hay build image mới.
- Tránh rebuild Docker image: Nếu config thay đổi (ví dụ: IP DB, API key không nhạy cảm, timeout...), bạn không phải rebuild và push image mới → tiết kiệm thời gian, giảm rủi ro.
- Dễ quản lý tập trung: Nhiều Pod/Deployment dùng chung một ConfigMap → thay đổi một chỗ ảnh hưởng toàn bộ.
- Hỗ trợ nhiều cách consume:
 - Environment variables (biến môi trường).
 - Command-line arguments.
 - Mount như volume (file config trong container).

Không có ConfigMap → config bị hardcode trong image → mỗi thay đổi nhỏ phải rebuild, deploy lại → không tối ưu cho production.

3. ConfigMap lưu trữ dữ liệu như thế nào?

ConfigMap có hai phần chính:

- data: Lưu dữ liệu dạng key-value (plain text) – ví dụ:
 - database_url: "mysql://prod-db:3306"
 - log_level: "debug"
- binaryData: Lưu dữ liệu nhị phân (base64-encoded) – ít dùng hơn.

Bạn có thể lưu:

- Các giá trị đơn lẻ (key-value).
- Toàn bộ file config (ví dụ: application.properties, config.json) – key là tên file, value là nội dung file.

4. Cách Pod/Deployment sử dụng ConfigMap (các cách phổ biến)

- Environment variables: Inject key-value từ ConfigMap thành biến môi trường trong container.
 - Volume mount: Mount ConfigMap như một thư mục volume → container thấy file config như file thật (ví dụ: /app/config/application.properties).
 - Command-line args: Truyền giá trị từ ConfigMap làm tham số khởi động container.
- Khi ConfigMap thay đổi → Pod không tự reload config (trừ khi app hỗ trợ hot-reload). Thường cần restart Pod (redeploy Deployment) để áp dụng thay đổi.

5. ConfigMap vs Secret (so sánh quan trọng – câu hỏi kinh điển phỏng vấn)

ConfigMap cho config "công khai", Secret cho config "riêng tư". Không nên lưu mật khẩu vào ConfigMap (dù base64 cũng không an toàn).

6. ConfigMap trong thực tế & phỏng vấn

- ConfigMap là namespaced (thuộc namespace) → tách biệt config giữa các dự án/môi trường.
- Không tự động reload: Thay đổi ConfigMap → cần restart Pod (redploy) để áp dụng (trừ app hỗ trợ watch config).
- Best practice:
 - Tách config theo môi trường (config-dev, config-prod).
 - Mount file config thay vì env vars nếu file lớn/complex.
 - Kết hợp với Deployment (không dùng Pod độc lập).
 - Không lưu bí mật → dùng Secret.
- Phỏng vấn hay hỏi: "Làm sao update config mà không rebuild image?" → Trả lời: Dùng ConfigMap + volume mount → update ConfigMap → redeploy Deployment.

7. Lưu ý thực tế

- ConfigMap giúp ứng dụng 12-factor app (config lưu ngoài code).
- Production: Kết hợp với tools như Helm (values.yaml) hoặc external config service (Consul, Vault) cho config động hơn.
- Không dùng ConfigMap cho dữ liệu lớn/binary → dùng volume khác.

Secret

1. Secret là gì? Định nghĩa đơn giản

Secret là một API object trong Kubernetes dùng để lưu trữ dữ liệu nhạy cảm (sensitive data) như mật khẩu, API key, token, chứng chỉ SSL, thông tin đăng nhập Docker registry...

Nó giống như ConfigMap nhưng dành cho dữ liệu bí mật:

- Dữ liệu được base64-encoded (mã hóa cơ bản) khi lưu trữ và truyền qua mạng.
- Không được ghi vào log hệ thống (etcd log, kubelet log...).
- Có cơ chế kiểm soát truy cập chặt chẽ hơn (RBAC hạn chế ai được đọc Secret).

Mục đích chính: Bảo vệ thông tin nhạy cảm khỏi lộ ra ngoài (ví dụ: không hardcode password vào Docker image, không lưu plaintext trong ConfigMap).

2. Tại sao cần Secret? (Mục đích chính – hay hỏi phỏng vấn)

- Bảo mật dữ liệu nhạy cảm: Không nên lưu password, token, private key trong ConfigMap (plaintext) hoặc Docker image → Secret giúp tách biệt và bảo vệ chúng.
- Tránh lộ thông tin:
 - Base64 không phải mã hóa thật (chỉ obfuscation), nhưng ngăn chặn lỗi định dạng khi truyền và hạn chế lộ trực tiếp trong log.
 - Hạn chế truy cập qua API (RBAC), etcd encryption (nếu bật).

- Linh hoạt như ConfigMap: Có thể inject vào Pod dưới dạng biến môi trường, volume mount, hoặc command args → thay đổi Secret mà không rebuild image.
 - Hỗ trợ nhiều loại dữ liệu bí mật: Không chỉ key-value, mà còn chứng chỉ, token, file .dockerconfigjson...
- Không có Secret → dễ lộ thông tin nhạy cảm (ví dụ: password DB trong image → nguy hiểm khi image public hoặc bị tấn công).

3. Các loại Secret phổ biến (hay hỏi phỏng vấn)

Kubernetes hỗ trợ nhiều type của Secret, mỗi loại có định dạng dữ liệu chuẩn:

- Opaque (mặc định, phổ biến nhất): Lưu trữ key-value tùy ý (password, API key, token...). Dùng stringData (plaintext) hoặc data (base64).
 - kubernetes.io/service-account-token: Token cho Service Account (dùng để Pod xác thực với API server, RBAC).
 - kubernetes.io/dockerconfigjson: Lưu thông tin đăng nhập Docker registry (tên user, password/token) dưới dạng file .dockerconfigjson – dùng để pull private image.
 - kubernetes.io/basic-auth: Chỉ lưu username + password (dùng cho basic authentication).
 - kubernetes.io/tls: Lưu certificate (crt) + private key (key) – dùng cho HTTPS/TLS termination.
 - bootstrap.kubernetes.io/token: Token dùng để join node mới vào cụm (bootstrap token).
- Thực tế:
- Opaque → dùng cho hầu hết password, key.
 - dockerconfigjson → pull image từ private registry.
 - tls → chứng chỉ HTTPS (Ingress, webhook...).

4. Cách Pod/Deployment sử dụng Secret (các cách phổ biến)

Giống ConfigMap:

- Environment variables: Inject giá trị từ Secret thành biến môi trường trong container (ví dụ: DB_PASSWORD).
 - Volume mount: Mount Secret như file (ví dụ: /etc/secret/tls.crt và tls.key).
 - Image pull secrets: Gắn Secret vào Pod/ServiceAccount để pull private Docker image.
- Khi Secret thay đổi → Pod không tự reload (trừ app hỗ trợ hot-reload). Thường cần restart Pod (redploy Deployment) để áp dụng.

5. Secret vs ConfigMap (so sánh – câu hỏi kinh điển phỏng vấn)

ConfigMap cho config "công khai", Secret cho config "riêng tư/bí mật". Không lưu password vào ConfigMap!

6. Secret trong thực tế & phỏng vấn

- Secret là namespaced (thuộc namespace) → tách biệt bí mật giữa các dự án/môi trường.

- Không an toàn tuyệt đối: Base64 chỉ obfuscation → dễ decode. Production nên bật etcd encryption at rest + RBAC nghiêm ngặt + external secret manager (Vault, AWS Secrets Manager, Azure Key Vault...).
- Best practice:
 - Dùng stringData khi tạo (Kubernetes tự base64).
 - Mount file thay vì env vars nếu dữ liệu lớn (ví dụ: cert file).
 - Không commit Secret vào Git (dùng Sealed Secrets hoặc external tools).
 - Kết hợp với Deployment (không dùng Pod độc lập).
- Phỏng vấn hay hỏi: "Làm sao lưu password DB an toàn?" → Trả lời: Dùng Secret loại Opaque + env vars/volume mount → tránh hardcode trong image.

7. Lưu ý thực tế

- Secret giúp tuân thủ 12-factor app (config/bí mật lưu ngoài code).
- Production: Không dùng Secret plaintext → kết hợp với HashiCorp Vault, External Secrets Operator.
- Không dùng Secret cho dữ liệu lớn → dùng PersistentVolume.

StorageClass

1. StorageClass là gì? Định nghĩa đơn giản

StorageClass là một API object trong Kubernetes dùng để định nghĩa các loại lưu trữ khác nhau (storage types) mà cụm có thể cung cấp cho PersistentVolume (PV) và PersistentVolumeClaim (PVC).

Nó giống như các loại kho hàng trong tiệm bánh:

- Kho lạnh (SSD, tốc độ cao) → dùng cho bánh kem cần bảo quản lạnh, truy cập nhanh.
 - Kho khô (HDD, dung lượng lớn) → dùng cho bánh mì khô, lưu lâu dài, không cần tốc độ cao.
 - Kho VIP (premium storage trên cloud) → dùng cho dữ liệu quan trọng, cần độ bền cao.
- StorageClass giúp Kubernetes biết: "Loại lưu trữ nào phù hợp với yêu cầu của ứng dụng?" (tốc độ, chi phí, độ bền, vị trí địa lý...).

2. Tại sao cần StorageClass? (Mục đích chính – hay hỏi phỏng vấn)

- Dynamic Provisioning (cấp phát tự động): Thay vì admin phải tạo PV thủ công mỗi khi cần, StorageClass + Provisioner sẽ tự động tạo PV khi có PVC yêu cầu → tiết kiệm thời gian, scale dễ dàng.
- Phân loại lưu trữ đa dạng: Một cụm có thể có nhiều loại lưu trữ (SSD nhanh, HDD rẻ, replicated cho HA, local disk, cloud block/file storage...). StorageClass giúp chọn đúng loại phù hợp với workload.
- Tách biệt yêu cầu lưu trữ khỏi triển khai: Developer chỉ cần khai báo PVC với StorageClass mong muốn → Kubernetes tự xử lý phần backend (on-premise hay cloud).

- Hỗ trợ cloud-native: Trên AWS/GCP/Azure, StorageClass dùng CSI driver (EBS, GCE PD, Azure Disk...) để tự động tạo volume từ cloud provider.
- Không có StorageClass → phải tạo PV thủ công (static provisioning) → không scale tốt, không tự động, quản lý thủ công phức tạp.

3. StorageClass hoạt động như thế nào? (Mô hình cơ bản)

StorageClass khai báo:

- Provisioner (ai chịu trách nhiệm tạo PV): Ví dụ nfs.csi.k8s.io (on-premise), ebs.csi.aws.com (AWS), pd.csi.storage.gke.io (GCP).
- Parameters (tùy chọn): Loại disk (gp3, io1), replication, IOPS...
- Volume Binding Mode: Immediate (gắn ngay khi PVC tạo) hoặc WaitForFirstConsumer (chờ Pod schedule rồi mới gắn – tốt cho topology-aware).
- Reclaim Policy (khi PVC xóa): Delete (xóa PV) hoặc Retain (giữ lại dữ liệu).

Flow điển hình (dynamic provisioning):

- Developer tạo PVC → chỉ định StorageClass (ví dụ: "fast-ssd").
- Kubernetes gọi Provisioner của StorageClass.
- Provisioner tạo PV (tự động trên cloud hoặc NFS server on-premise).
- PV được bind vào PVC → Pod mount được volume.

Ví dụ tiệm bánh:

- StorageClass = "kho lạnh" (SSD nhanh).
- PVC = "tôi cần kho lạnh để lưu bánh kem".
- Provisioner = tự động mở kho lạnh mới và cấp cho PVC.

4. StorageClass vs PV vs PVC (so sánh – câu hỏi kinh điển phỏng vấn)

StorageClass = "menu kho hàng", PV = "kho đã thuê", PVC = "đơn đặt hàng kho".

5. StorageClass trong thực tế & phỏng vấn

- On-premise: Thường dùng NFS (nfs.csi.k8s.io), GlusterFS, Ceph RBD, Longhorn... → không có provisioning sẵn, cần cài Provisioner thủ công.
- Cloud (AWS/GCP/Azure): Có sẵn CSI driver + StorageClass mặc định (ví dụ: gp3, standard, premium SSD) → dynamic provisioning dễ dàng.
- Volume Binding Mode:
 - Immediate (mặc định): Bind ngay → có thể lãng phí nếu Pod không schedule.
 - WaitForFirstConsumer: Bind sau khi Pod schedule → tối ưu topology (ví dụ: gắn volume cùng zone với Pod).
- Reclaim Policy: Delete (tự xóa dữ liệu khi PVC xóa – phổ biến dev/test), Retain (giữ lại – production cần backup).

Phỏng vấn hay hỏi: "Dynamic provisioning là gì?" → Trả lời: StorageClass + Provisioner tự động tạo PV khi có PVC → không cần admin tạo thủ công.

6. Lưu ý thực tế

- StorageClass là cluster-scoped (toàn cụm, không thuộc namespace).
- Production: Luôn dùng dynamic provisioning + CSI driver (Container Storage Interface) – chuẩn hiện đại.
- Best practice: Đặt tên StorageClass tường minh (fast-ssd-prod, standard-hdd-dev) và dùng WaitForFirstConsumer cho cloud multi-zone.
- Không dùng local volume cho production (không HA) → ưu tiên replicated storage (Ceph, Longhorn, cloud block storage).

Persistent Volume & Persistent Volume Claim

1. PersistentVolume (PV) là gì? Định nghĩa đơn giản

PersistentVolume (PV) là một API object trong Kubernetes đại diện cho một vùng lưu trữ bền vững (persistent storage) đã được cấp phát sẵn trong cụm.

Nó giống như một cái tủ kho đã có sẵn trong tiệm bánh:

- Tủ đã được chuẩn bị (10 GB, 20 GB...), có thể chứa bánh (dữ liệu).
- Tủ này tồn tại độc lập, không phụ thuộc vào Pod nào đang dùng nó.
- PV có thể dựa trên nhiều backend: NFS, local disk, cloud block storage (AWS EBS, GCP Persistent Disk), Ceph, GlusterFS...

PV là tài nguyên cluster-wide (toàn cụm), do admin/DevOps tạo (static) hoặc tự động tạo bởi StorageClass (dynamic).

2. PersistentVolumeClaim (PVC) là gì? Định nghĩa đơn giản

PersistentVolumeClaim (PVC) là một yêu cầu sử dụng lưu trữ từ phía ứng dụng (developer).

Nó giống như nhân viên yêu cầu tủ kho:

- "Tôi cần một tủ 5 GB để chứa nguyên liệu bánh kem".
- PVC khai báo: dung lượng cần (storage request), access mode (ReadWriteOnce, ReadWriteMany...), StorageClass (loại tủ).
- Kubernetes sẽ tìm PV phù hợp (đáp ứng yêu cầu) và bind (gắn) PV vào PVC → Pod mount được volume.

PVC thuộc namespace (riêng cho từng dự án/môi trường), giúp developer không cần quan tâm backend lưu trữ là gì.

3. PV và PVC hoạt động như thế nào? (Mô hình cơ bản)

- Static Provisioning (thủ công): Admin tạo PV trước → PVC yêu cầu → Kubernetes bind nếu khớp (dung lượng, access mode, StorageClass...).
- Dynamic Provisioning (tự động – phổ biến production): PVC yêu cầu StorageClass → Provisioner (từ StorageClass) tự tạo PV mới → bind ngay.

Ví dụ tiệm bánh:

- PV = cái tủ kho đã có sẵn (10 GB, NFS).
 - PVC = nhân viên yêu cầu "tôi cần tủ 5 GB, loại NFS nhanh".
 - Kubernetes = quản lý tiệm → tìm tủ phù hợp (hoặc mở tủ mới nếu dynamic) và giao cho nhân viên.
 - Pod = nhân viên → mount tủ vào quầy (container) để lưu bánh (dữ liệu).
- Khi Pod chết hoặc scale → dữ liệu vẫn an toàn trong PV (không mất như emptyDir).

4. Các khái niệm quan trọng của PV & PVC (hay hỏi phỏng vấn)

Access Modes (chế độ truy cập):

- ReadWriteOnce (RWO): Chỉ 1 Node đọc/ghi được (phổ biến nhất cho block storage như EBS, local disk).
- ReadOnlyMany (ROX): Nhiều Node đọc, không ghi (dùng cho nội dung tĩnh).
- ReadWriteMany (RWX): Nhiều Node đọc/ghi cùng lúc (dùng cho NFS, CephFS, EFS – file storage).

Reclaim Policy (chính sách thu hồi khi PVC xóa):

- Delete: Xóa PV và dữ liệu (mặc định dynamic provisioning trên cloud).
- Retain: Giữ PV và dữ liệu lại (admin phải dọn thủ công – an toàn production).
- Recycle (cũ, ít dùng): Xóa dữ liệu và tái sử dụng PV (chỉ hỗ trợ NFS/hostPath).

Volume Mode:

- Filesystem (mặc định): Mount như thư mục.
- Block: Mount như thiết bị block (raw device – dùng cho database cần performance cao).

Phase của PV: Available → Bound → Released → Failed.

Phase của PVC: Pending → Bound.

5. PV vs PVC vs StorageClass (so sánh – câu hỏi kinh điển)

StorageClass = "menu tủ kho", PV = "tủ đã có sẵn", PVC = "đơn đặt tủ".

6. PV & PVC trong thực tế & phỏng vấn

- HostPath: Chỉ dùng test/dev (lưu trực tiếp trên Node) → không bền vững (Pod di chuyển Node → mất dữ liệu). Không dùng production.
- NFS: RWX phổ biến on-premise → nhiều Pod đọc/ghi cùng lúc (nhưng không phải HA thực thụ).
- Cloud: Dynamic provisioning với CSI driver (EBS, GCE PD, Azure Disk) → tự động, dễ scale.
- StatefulSet thường dùng PVC (mỗi Pod có PVC riêng, giữ identity và dữ liệu).
- Deployment (stateless) ít dùng PVC (dùng ConfigMap/Secret hoặc external DB).
- Phỏng vấn hay hỏi: "Làm sao đảm bảo dữ liệu không mất khi Pod chết?" → Trả lời: Dùng PV + PVC + StorageClass → dữ liệu lưu bền vững ngoài Pod lifecycle.

7. Lưu ý thực tế

- PV/PVC là nền tảng cho stateful workload (database, Redis, Elasticsearch...).
- Production: Ưu tiên dynamic provisioning + RWX nếu cần shared storage, hoặc RWO với replicated backend (Ceph, Longhorn).
- Luôn dùng Retain reclaim policy cho dữ liệu quan trọng → tránh mất mát khi xóa nhầm PVC.

HPA (Horizontal Pod Autoscaler)

1. HPA là gì? Định nghĩa đơn giản

Horizontal Pod Autoscaler (HPA) là một controller trong Kubernetes tự động tăng/giảm số lượng Pod (replicas) của một Deployment/StatefulSet dựa trên tải thực tế (metrics như CPU, memory, hoặc custom metrics).

Nó giống như người quản lý quầy bánh thông minh:

- Bình thường chỉ cần 2 nhân viên (2 Pod) để phục vụ 10.000 khách đồng thời (CCU).
- Khi có sự kiện, khách tăng đột biến lên 100.000 CCU → HPA tự động gọi thêm nhân viên (scale lên 6–10 Pod) để không bị chậm hoặc down.
- Khi hết giờ cao điểm, khách giảm → HPA tự động cho nhân viên nghỉ (scale xuống còn 2 Pod) để tiết kiệm tài nguyên.

HPA thực hiện horizontal scaling (mở rộng theo chiều ngang – thêm Pod), khác với vertical scaling (tăng CPU/RAM cho Pod hiện tại).

2. Tại sao cần HPA? (Mục đích chính – hay hỏi phỏng vấn)

- Tự động thích ứng tải biến động: Đáp ứng traffic/transaction tăng đột biến (ví dụ: giờ đăng ký tin chỉ trường học, Black Friday, giờ cao điểm giao dịch ngân hàng) mà không cần can thiệp thủ công.
- Đảm bảo tính sẵn sàng (availability): Giữ latency thấp, tránh down khi quá tải.
- Tối ưu chi phí: Chỉ scale up khi cần (giờ cao điểm), scale down khi không cần → tiết kiệm tài nguyên server/cloud.
- Kết hợp với metrics: Dựa trên CPU/memory usage (mặc định) hoặc custom metrics (Kafka lag, transaction per second, queue length...) để quyết định scale.

Không có HPA → phải scale thủ công (kubectl scale) → chậm, dễ miss peak time, tốn kém nếu scale dư thừa.

3. HPA hoạt động như thế nào? (Mô hình cơ bản)

HPA cần 2 điều kiện tiên quyết:

- Metrics Server (hoặc Prometheus + Adapter) để thu thập CPU/memory usage của Pod.
- Deployment/StatefulSet đã khai báo requests & limits (CPU/memory) – HPA dựa vào đó để tính % sử dụng.

Quy trình:

1. HPA theo dõi metrics liên tục (mặc định mỗi 15–30 giây).
2. Nếu CPU usage > target (ví dụ: 50–80% của limit) → scale up (tăng replicas).
3. Nếu usage thấp lâu (mặc định 5 phút) → scale down (giảm replicas về min).
4. Giới hạn: minReplicas (thường 1–2) và maxReplicas (ví dụ 10–50 tùy dự án).

Ví dụ thực tế:

- Deployment có 2 Pod, mỗi Pod limit CPU 1 core.
- HPA target CPU utilization = 60%.
- Khi tổng CPU usage > 60% (tức > 1.2 core cho 2 Pod) → HPA scale lên 4 Pod.
- Khi usage giảm < 60% lâu → scale xuống 2 Pod.

4. Các metrics HPA sử dụng (hay hỏi phỏng vấn)

- Resource metrics (mặc định):
 - CPU utilization (phần trăm so với request/limit).
 - Memory utilization (ít dùng hơn vì memory scale khó dự đoán).
- Custom metrics (rất phổ biến production):
 - Transactions per second (TPS).
 - Kafka consumer lag.
 - Queue length (RabbitMQ, Redis).
 - HTTP requests per second.
- External metrics (từ bên ngoài): Prometheus, Datadog, New Relic...

Production banking thường dùng custom metrics (TPS, latency) thay vì chỉ CPU.

5. HPA vs Vertical Scaling (so sánh – câu hỏi kinh điển)

HPA là lựa chọn ưu tiên cho microservices (như gateway banking của bạn).

6. HPA trong thực tế & phỏng vấn

- Min/Max replicas: Min thường 2–3 (HA), max tùy dự án (10–100) – cần test để tránh scale quá đà.
- Scale down delay: Mặc định 5 phút → tránh flap (scale lên xuống liên tục).
- Cool down period: Scale up nhanh (15–30s), scale down chậm → an toàn.
- Production banking:
 - Kết hợp HPA + custom metrics (TPS, latency) để scale theo transaction rate.
 - Tránh scale quá nhanh → có thể gây overload DB/Kafka.
 - Thường dùng cùng Cluster Autoscaler (scale Node) để mở rộng hạ tầng.
- Phỏng vấn hay hỏi:
 - “HPA scale dựa trên gì?” → CPU/memory hoặc custom metrics.
 - “Làm sao tránh flap khi scale?” → Scale down delay + stabilization window.

- “Khi nào không nên dùng HPA?” → Stateful app không scale ngang tốt, hoặc tải ổn định không biến động.

7. Lưu ý thực tế

- HPA chỉ scale Pod, không scale Node → cần Cluster Autoscaler nếu dùng cloud (EKS, GKE, AKS).
- Không scale dựa chỉ CPU nếu app bị bottleneck I/O hoặc DB → dùng custom metrics.
- Test kỹ: Load test (Locust, JMeter) để tìm ngưỡng target CPU hợp lý.
- Banking: Kết hợp HPA + circuit breaker + rate limiting (Ingress) để bảo vệ hệ thống.